

Flash Attention

Apar Pokhrel Che Kwanga Pisit Nakhonekhong Kopil Sharma
Jose Lopez

May 17, 2026

Abstract

Transformer-based models have become the foundation of modern deep learning systems, but their scalability is often limited by the quadratic memory and compute complexity of the attention mechanism. In this project, we implement and optimize a CUDA-based variant of FlashAttention, a memory-efficient exact attention algorithm that reduces high-bandwidth memory access by tiling computations and leveraging on-chip GPU memory.

We evaluate our implementation against standard attention kernels and demonstrate significant improvements in both runtime performance and memory efficiency across varying sequence lengths and embedding dimensions. Experimental results show that our CUDA implementation achieves substantial speedups while maintaining exactness, enabling scalable training and inference for large transformer models on resource-constrained hardware. This work highlights the importance of memory-aware kernel design in bridging the gap between theoretical efficiency and practical deployment of attention mechanisms.

1 Standard Attention

The attention mechanism is a core element that sits at the heart of transformers like GPT[1], LLaMA, BERT, and Mixture of Experts (MoE).

Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ where N is the sequence length and d is the head dimension, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where softmax is applied row-wise.

Standard attention implementations materialize the matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM, which takes $O(N^2)$ memory. Often $N \gg d$ (e.g., for GPT2, $N = 1024$ and $d = 64$). From the paper[2], the standard attention implementation is described in Algorithm 0.

Algorithm 0 Standard Attention Implementation

Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

1.1 Why is standard implementation slow?

The main reasoning is the amount of memory access. Standard attention mechanism uses High Bandwidth Memory (HBM) to store, read and write keys, queries and values. HBM is large in memory, but slow in processing, meanwhile SRAM is smaller in memory, but faster in operations. In the standard attention implementation, the cost of loading and writing keys, queries, and values from HBM is high. It loads keys, queries, and values from HBM to GPU on-chip SRAM, performs a single step of the attention mechanism, writes it back to HBM, and repeats this for every single attention step. This process is clearly demonstrated in Fig. 1

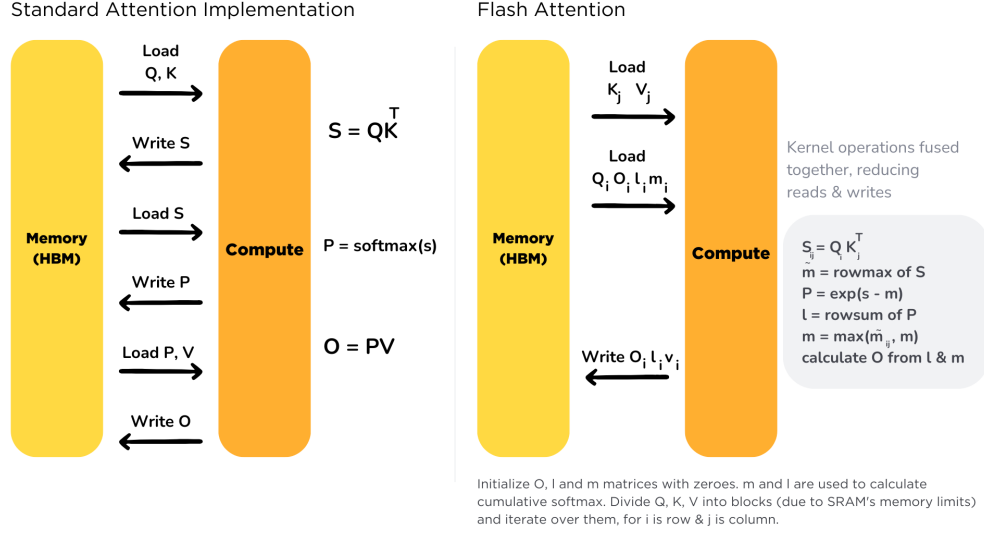


Figure 1: Stanard vs Flash Attention

2 Flash Attention

FlashAttention is a revolutionary technique that leverages efficient I/O awareness to reduce overhead in computing the final attention matrix. FlashAttention achieves this by restructuring the computation using two techniques: **tilting** and **online softmax**. In this report, we only focus on the forward pass and leave the backward pass for further reading. Fig. 4 borrowed from the main paper, shows the overall process of how FlashAttention works.

2.1 Algorithm and Analysis

Algorithm 1 gives a detailed overview of the computations involved in FlashAttention. Given the inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, the main goal is to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$ and write it to HBM such that it reduces the amount of HBM accesses (to sub-quadratic in N).

The inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are split into blocks, loaded from the slow HBM to fast SRAM, then attention output is computed with respect to those blocks.

Tiling: The computation is broken into blocks. The matrices are partitioned into smaller sub-matrices. The algorithm iterates through blocks of $\mathbf{K}$ and $\mathbf{V}$, loading them into SRAM. Within

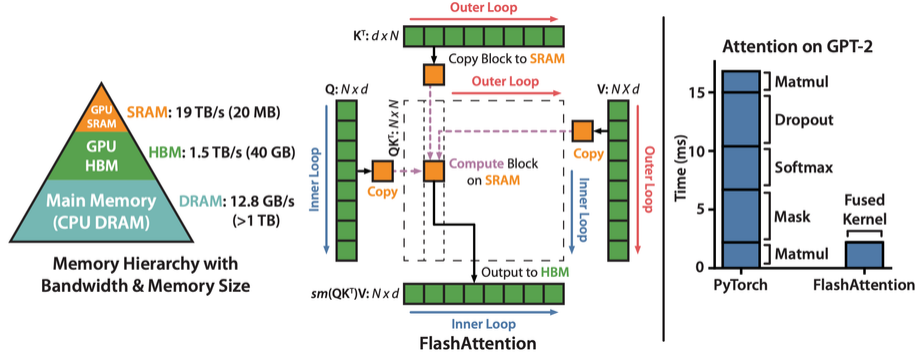


Figure 2: Overall working of Flashattention

this outer loop, it iterates through blocks of Q .

Basically, attention is calculated over the full full sequence by processing tokens in small portions.

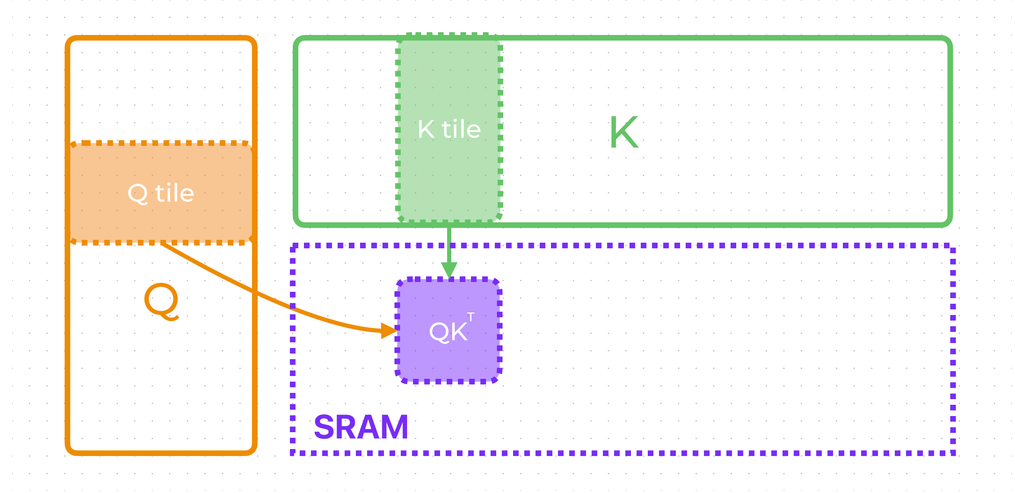


Figure 3: Tiling over Q and K

As shown above, the tiled structure changes the memory access pattern.

Online Softmax: All that is remaining is to apply softmax, multiply by V to get the output matrix. The softmax function is applied row-wise. For a row i of the attention score matrix S , the output is

$$P_{ij} = \frac{e^{S_{ij}}}{\sum_k e^{S_{ik}}}.$$

This requires aggregation over the sequence length to normalize the scores. Due to our tiling approach, we do not have access to the whole length as we load the data in tiles.

This is resolved by using the online softmax algorithm. It maintains two running statistics for each row i : the maximum value seen so far, denoted m_i , and the sum of exponentials of the scaled values, denoted ℓ_i . For numerical stability, the softmax is computed as

$$\text{softmax}(x_j) = \frac{e^{x_j - m}}{\sum_k e^{x_k - m}}, \quad m = \max_k x_k.$$

When processing the j -th block of a row x_i , which we denote $x_{i,j}$, we can update the statistics and the output O_i as follows:

1. Compute the local statistics for the current block:

$$m_{i,j} = \max_k (x_{ik})$$

and

$$\ell_{i,j} = \sum_k e^{x_{ik} - m_{i,j}}.$$

2. Compute the new global maximum:

$$m_i^{\text{new}} = \max(m_i^{\text{old}}, m_{i,j}).$$

3. Update the output value using the old and new statistics:

$$O_i^{\text{new}} = \frac{1}{\ell_i^{\text{new}}} \left(\ell_i^{\text{old}} e^{m_i^{\text{old}} - m_i^{\text{new}}} O_i^{\text{old}} + \ell_{i,j} e^{m_{i,j} - m_i^{\text{new}}} O_{i,j} \right).$$

where $O_{i,j}$ is the attention output computed using only the current block of K and V , and ℓ_i^{new} is the appropriately rescaled sum.

FlashAttention computes the exact softmax output without requiring the entire row to reside in memory at once by storing and updating only the two scalars m_i and ℓ_i for each row.

Algorithm 1

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
 - 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 5: **for** $1 \leq j \leq T_c$ **do**
 - 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 7: **for** $1 \leq i \leq T_r$ **do**
 - 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
 - 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
 - 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
 - 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
 - 14: **end for**
 - 15: **end for**
 - 16: Return $\mathbf{O}$.
-

3 Repository Structure

Our code repository is available [here](#). The repository is organized into several components.

Under the `FlashAttention/src`,

- **Dataset Generator:** Produces 8 different sets Q , K , and V inputs along with expected outputs.
- **Build System (CMake):** Defines three targets:
 - `fa_test` — correctness validation
 - `fa_datagen` — dataset generation
 - `fa_benchmark` — performance evaluation
- **CUDA:**
 - `fa_cpu` — CPU version of attention
 - `fa_gpu` — global and shared memory of FlashAttention
- **Triton Implementation:** The `triton/` directory contains a reference-style FlashAttention implementation for comparison borrowed from several online sources.

4 Procedure

The testing pipeline is designed to ensure both correctness and consistency across implementations:

- **Generate Test Data:** Input matrices Q , K , and V are generated with controlled dimensions and values. The same datasets are used across all implementations to ensure a fair comparison.
- **Reference Execution (CPU):** A CPU-based attention implementation is executed first to produce a ground-truth output. This serves as the reference baseline for validation.
- **GPU Implementations:** Two GPU-based approaches are evaluated:
 - Global memory implementation
 - Shared memory (tiled / FlashAttention-style) implementation

Each implementation processes identical inputs to ensure consistency.

- **Output Comparison:** The resulting output matrices are compared element-wise against the CPU reference using a predefined numerical tolerance (e.g., based on absolute or relative error) to account for floating-point differences.

4.1 CPU Baseline Implementation

To establish a foundation for correctness and performance comparison, we first developed a CPU reference implementation. This version computes attention scores using dot products between rows of Q and K , applies the softmax function row by row, and multiplies the result with V to produce the final output.

While this baseline is simple, readable, and useful for verifying the mathematical correctness of later implementations, it is severely limited in performance. The computation is executed serially using nested loops on a single CPU core, and it necessitates storing the full $N \times N$ attention score matrix in memory, making it extremely slow for practical workloads.

4.2 Naive GPU Implementation

To improve upon the slow CPU baseline, we developed a naive GPU implementation utilizing CUDA to leverage massive parallelization. By moving the attention computation to the GPU, many threads can compute matrix elements simultaneously, resulting in a much faster execution time than the CPU version.

However, despite this significant computational speedup, a major memory bottleneck remains. This naive approach still depends heavily on slow global memory and stores the full $N \times N$ attention matrix during computation. Consequently, the memory bandwidth becomes the primary bottleneck that limits overall performance.

4.3 Hardware Memory Hierarchy and the Shared Memory Solution

To overcome the global memory bottleneck observed in the naive implementation, it is important to understand the GPU’s memory hierarchy and its impact on bandwidth. The GPU’s main global memory, or High Bandwidth Memory (HBM), possesses a large capacity (e.g., 40 GB) but operates with a relatively slow bandwidth of approximately 1.5 TB/s [2]. In contrast, on-chip shared memory (SRAM) is much smaller (e.g., 20 MB) but significantly faster, offering read/write bandwidths up to 19 TB/s [2].

Standard implementations are constrained because they repeatedly read and write intermediate matrices to the slow HBM. By actively utilizing the faster shared memory, we can drastically reduce the reliance on global memory access. This forms the basis for our optimized FlashAttention approach, which loads data in smaller blocks to ensure that the heaviest computational loops are performed entirely within the faster SRAM before writing the final output back to the HBM.

4.4 Results and Analysis

Both GPU implementations produce outputs that match the CPU reference within the specified numerical tolerance, confirming correctness.

The global memory implementation achieves speedup due to parallel execution on the GPU, but still incurs higher memory access latency. In contrast, the shared memory implementation further improves performance by reducing global memory accesses and reusing data through tiling.

Overall, the shared memory (FlashAttention-style) approach provides the best performance while maintaining numerical stability and accuracy comparable to the CPU baseline.

5 Execution Time & Speedup

The following two tables summarize the measured execution times and speed up for each implementation:

CPU Implementation (Slowest): This version relied on nested loops where each operation was executed in a strictly sequential manner. Because CPU architectures are optimized for complex branching and low-latency single-thread performance rather than massive throughput, the $O(n^2)$

Implementation	Execution Time (ms)
CPU Attention	5831
GPU Global Memory	175
GPU Shared Memory	18

Table 1: Measured execution times and speedup across implementations.

Implementation	Speedup
Global vs CPU	33x
Shared vs CPU	322x
Shared vs Global	9.7x

```

Success: GPU Global Memory vs CPU Attention Map matrices match within tolerance 0.000010
Success: GPU Shared Memory vs CPU Attention Map matrices match within tolerance 0.000010
Success: GPU Global Memory vs CPU Output matrices match within tolerance 0.000010
Success: GPU Shared Memory vs CPU Output matrices match within tolerance 0.000010

CPU Execution Time: 5831.061 ms
GPU Global Execution Time: 175.323 ms
GPU Shared Execution Time: 18.111 ms
aparpokhre@gmclab:~/cuda-flashattention/FlashAttention$

```

Figure 4: Execution of experiments

complexity of the attention mechanism place a tremendous load on the processor. Furthermore, the CPU’s L1/L2 caches are managed by hardware; in this implementation, the frequent, repetitive fetching of matrix elements from DRAM (Main Memory) resulted in high cache-miss rates and significant idle cycles while waiting for data.

GPU Global Memory Version (Faster): By porting the kernel to CUDA, the computation was decomposed into thousands of individual threads, resulting in a 33× speedup. This leap is due to the GPU’s ability to hide latency by scheduling other warps while one warp waits for a memory fetch. However, this version was heavily "Memory Bound." Since every thread had to fetch its required data directly from Global Memory (VRAM), the high latency of these transactions (typically 400–800 clock cycles) and the limited bus bandwidth prevented the CUDA cores from reaching maximum utilization.

GPU Shared Memory Version (Fastest): The optimized version achieved an additional 9.7× speedup over the global memory version by utilizing the programmable L1 cache, known as **Shared Memory**. By implementing a "tiling" strategy, threads within a block collaborated to load a block of data into on-chip shared memory once. Once loaded, all subsequent calculations for that tile accessed the data at near-register speeds (approx. 100× faster than global memory). This drastically reduced the total number of VRAM transactions and allowed the GPU to operate closer to its theoretical compute peak.

6 Project Challenges

Throughout the development process we faced multiple technical challenges while implementing our flash attention inspired optimization.

- **Understanding Attention Complexity:** Attention sounds like it is just simple matrix

multiplication. It involves additional steps such as softmax normalization and requires careful handling of numerical stability. It took us some time to figure it out.

- **Floating-Point Precision:** CPU and GPU calculations can produce slightly different floating-point results because they may execute operations in different orders or with different precision behavior.
- **Team Integration:** Separate parts of the project, such as the CPU implementation, CUDA kernels, and benchmarking code, were worked on by different team members. To integrate it all at last was pretty challenging.
- **Build Configuration:** Proper configuration of the build system using CMake and correct specification of paths (e.g., `GPULK_INSTALL_PATH`) was necessary for successful compilation. The CMake gave us trouble it does always.

7 Limitations of Our Implementation

While we are successfully able to demonstrate the core concept of how Flash attention works, we still have lots of limitations to our implementation.

- **Core Concept Demonstration:** We tried to implement or illustrate the main idea of reducing bottlenecks in attention computation. But it was not a full production level implementation like in the official flash attention library which required a lot of time to replicate.
- **Limited Test Scope:** We didn't have much test values to perform the evaluation on wide range of input sizes. We used limited range of input sizes. Real-world transformer models often use much larger sequence lengths, so performance may vary at scale.
- **Missing Advanced Optimizations:** Our implementation was still missing some advanced optimizations such as full kernel fusion, highly optimized tiling strategies, and hardware-specific tuning to help it adjust to different GPU devices.
- **Hardware-Dependent Results:** GPU architecture might influence the performance of our work. Different GPUs may produce different speedups due to variations in memory bandwidth and compute capabilities.

8 Conclusion

In this project, we implemented and compared CPU, naive GPU, and optimized GPU versions of the attention mechanism.

- **Implementation Success:** The CPU implementation was a slow baseline but naive CUDA implementation was seen to be a little faster because of the parallelism. The best performance was demonstrated by memory-optimized FlashAttention version.
- **Correctness Validated:** GPU and CPU outputs didn't match exactly but they did match within a very small tolerance. We were able to verify that all the versions that we implemented were correct.

- **Key Finding:** The results demonstrate that reducing memory movement is very important for improving performance. GPU does provides significant speedup but memory aware optimization, such as using shared memory and reducing global memory access performs even better than all of them.
- **Final Insight:** The effectiveness of the FlashAttention can be blamed upon its optimization of memory efficiency and how it is computed on the GPU hardware.

References

- [1] A. Vaswani et al., *Attention is all you need*, 2023. arXiv: 1706.03762 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/1706.03762>.
- [2] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35, Curran Associates, Inc., 2022, pp. 16 344–16 359. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf.